



NUST CHIP DESIGN CENTRE

## **RISC-V Architecture**

### **Lab Manual # 19**

## **RISC-V Datapath Implementation** **(Loads, Stores and Jumps)**

|                          |                                        |
|--------------------------|----------------------------------------|
| <b><u>Name</u></b>       | <i><u>Muddassir Ali Siddiqui</u></i>   |
| <b><u>Instructor</u></b> | <i><u>Sir Imran &amp; Sir Umer</u></i> |
| <b><u>Date</u></b>       | <i><u>28<sup>th</sup> Aug 2025</u></i> |

[. Top](#)

```
.text
.globl main

main:
    j first

second:

    addi x12, x0, 0xDE    # x12 = 125
    slli x12, x12, 8

    addi x12, x12, 0xAD    # x12 = 125
    slli x12, x12, 8

    addi x12, x12, 0xBE    # x12 = 125
    slli x12, x12, 8

    addi x12, x12, 0xAF    # x12 = 125
    j end

first:

    addi x12, x0, 0x12    # x12 = 125
    slli x12, x12, 8

    addi x12, x12, 0x34    # x12 = 125
    slli x12, x12, 8

    addi x12, x12, 0x56    # x12 = 125
    slli x12, x12, 8

    addi x12, x12, 0x78    # x12 = 125
    j second
```

```

j second

    li x1, 0x100    # load address of mybyte into x1
    sw x12, 0(x1)   # store byte from x12 into mybyte

    lw x2 0(x1)

    lb x2 0(x1)
    lb x3 1(x1)
    lb x4 2(x1)
    lb x5 3(x1)

end:

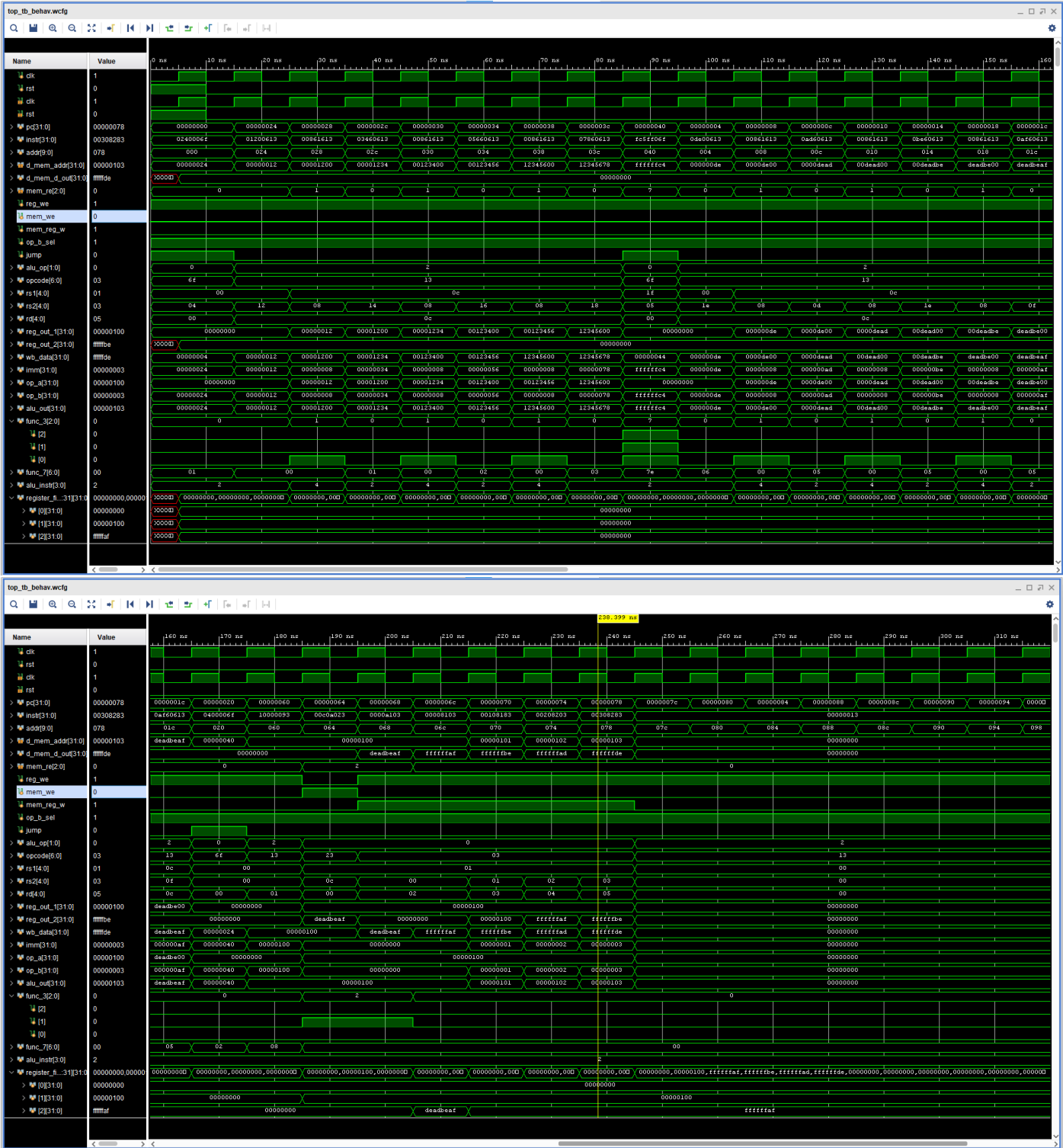
    li x1, 0x100    # load address of mybyte into x1
    sw x12, 0(x1)   # store byte from x12 into mybyte

    lw x2 0(x1)

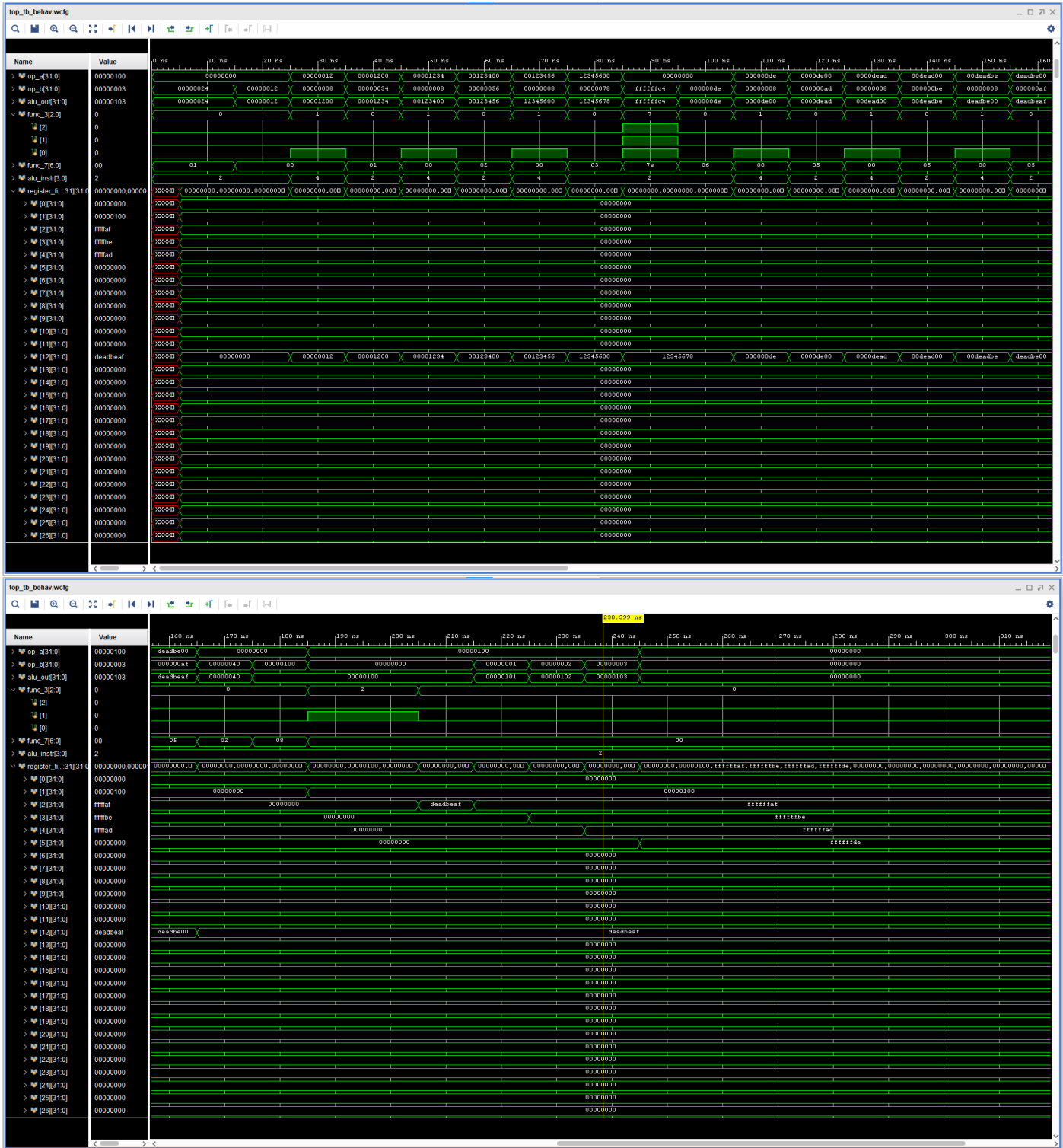
    lb x2 0(x1)
    lb x3 1(x1)
    lb x4 2(x1)
    lb x5 3(x1)

```

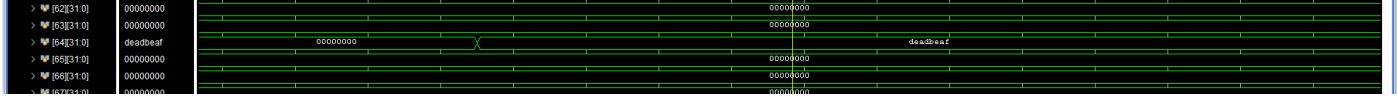
iii. Waveforms



Register File Output:



Data Memory:



### iv. Venus Results

|          |              |
|----------|--------------|
| a2 (x12) | 0x12345678   |
| a2 (x12) | 0xDEADBEAF   |
| ra (x1)  | 0x00000100   |
| sp (x2)  | 0xFFFFFFFFAF |
| gp (x3)  | 0xFFFFFFFFBE |
| tp (x4)  | 0xFFFFFFFFAD |
| t0 (x5)  | 0xFFFFFFFFDE |

## 2. Critical Analysis: (Write you critical analysis / conclusion here)

In this lab, I implemented a simplified RV32I core in SystemVerilog with support for load/store instructions (LW, SW) and jump operations (JAL, JALR). The design integrates the program counter, instruction memory, register file, ALU, immediate generator, data memory, and control unit into a functional datapath. A key challenge was handling the data memory, which is byte-addressable but word-aligned, ensuring correct access during load and store operations. Correct write-back logic was developed to handle ALU results, memory outputs, and PC+4 for jumps. The program counter update mechanism with pc\_sel and jump signals enables proper sequential execution and jump handling. Overall, the lab demonstrates the successful execution of a functional subset of the RV32I ISA, providing a solid base for extending the processor further.